

### 1 - O que é programação Orientada a objetos

É um jeito de programar que usa "objetos" pra organizar o código. A gente pensa nos problemas como se fossem coisas do mundo real, com características e ações próprias. Por exemplo, em vez de um monte de função solta, a gente cria um objeto "Carro" que tem cor, marca e consegue acelerar e frear.

### 2 - Quais são as vantagens em utilizar POO em relação a programação estruturada?

A principal vantagem é que o código fica muito mais organizado e bem fácil de entender, porque ele imita o mundo real. Além disso, com a POO, a gente consegue reutilizar código mais facilmente, o que economiza tempo. Também fica mais simples de dar futuras manutenção e adicionar coisas novas no programa sem quebrar o que já existe.

### 3 - Defina o conceito de objeto no paradigma POO

Objeto é a peça fundamental da POO. Pensamos nele como uma "coisa" que existe no nosso programa. Todo objeto tem duas principais coisas:

**Atributos:** são as características do objeto (exemplo: a cor de um carro, o nome de uma pessoa).

**Métodos:** são as ações que o objeto pode fazer (ex: um carro pode acelerar, uma pessoa pode andar, o carro pode frear, etc).

Basicamente, um objeto é uma instância de uma classe, que funciona como um molde para criar vários objetos parecidos.

### 4 - Explique os 4 principais fundamentos do paradigma POO

Os quatro pilares de POO são:

**1. Abstração:** É a ideia de focar no que é essencial. A gente simplifica as coisas complexas do mundo real, pegando só as características importantes para o nosso programa. Por exemplo, para um sistema de uma concessionária, o objeto "Carro" precisa ter cor, modelo e ano, mas não precisa ter tipo de pneu, apenas o que realmente precisa.

**2. Encapsulamento:** É como se a gente "escondesse" os detalhes de como um objeto funciona. Os atributos de um objeto ficam protegidos e só podem ser alterados pelos seus próprios métodos. Isso evita que alguém bagunce os dados do objeto sem querer e deixa o código mais seguro.

**3. Herança:** É a capacidade de uma classe "herdar" características e comportamentos de outra. Por exemplo, a gente pode ter uma classe "Veiculo" e depois criar as classes "Carro" e "Moto" que herdam tudo de "Veiculo" e adicionam suas próprias particularidades, isso evita repetir código já escrito anteriormente.

**4. Polimorfismo:** Significa "várias formas". Na prática, quer dizer que objetos de classes diferentes podem responder à mesma mensagem (ou método) de maneiras diferentes. Por exemplo, tanto um objeto "Cachorro" quanto um "Gato" podem ter um método emitirSom(),

mas o cachorro vai latir e o gato vai miar, mas ambos irão compartilhar a emissão de som, isso deixa o código mais flexível.